Unsupervised learning: Expectation Maximization (EM)

CSCI-P 556

ZORAN TIGANJ

Today: unsupervised learning

- Last time we covered clustering algorithm k-means.
- Today we will talk about Gaussian mixture models and how they can be used for clustering.
- We will cover Expectation Maximization algorithm which has many similarities with k-means:
  - It also initializes the cluster parameters randomly,
  - then it repeats two steps until convergence:
    - first assigning instances to clusters (this is called the expectation step)
    - then updating the clusters (this is called the maximization step).

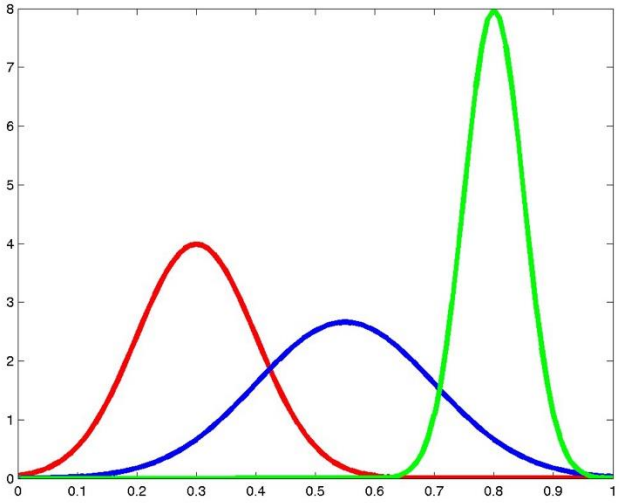
Gaussian mixture model

- A Gaussian mixture model (GMM) is a probabilistic model that assumes that the instances were generated from a mixture of several Gaussian distributions whose parameters are unknown.

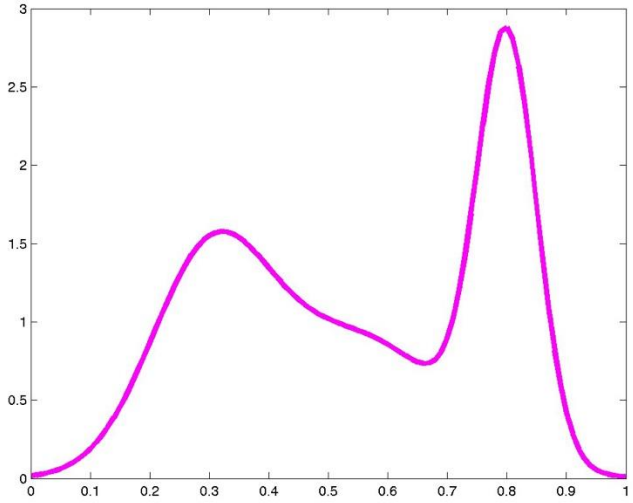
Mixture of Gaussians

- Imagine if we had data that is modeled by 3 different Gaussians
  - $X_1,\ldots X_N \sim N(0.3,0.2)$
  - $X_{N+1},\ldots X_M \sim N(0.55, 0.3)$
  - $X_{M+1},\ldots,X_K \sim N(0.8,0.1)$

- Suppose that this data is combined into one vector and we lose how each sample is distributed

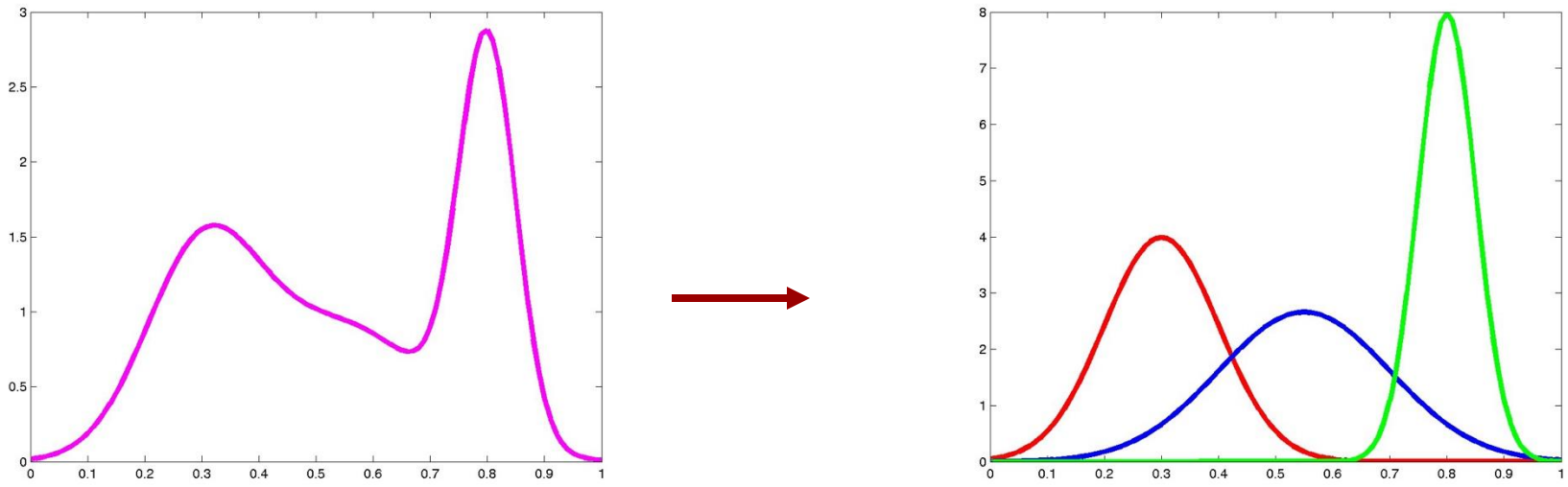


Data Histogram



Mixture of Gaussians

- Given the combined dataset without corresponding distributions
  - Can we recover the underlying Gaussians given the data?
  - Assuming, each data point is "generated" by one of the Gaussians



Mixture of Gaussians

- MOG is a small Bayesian network, where the mixture (or Gaussian) is a hidden (or latent) variable

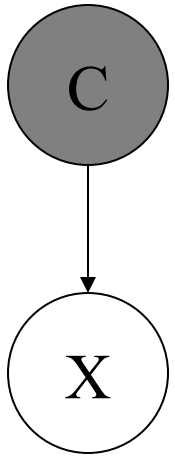
Probabilistic graphical model a probabilistic model for which a graph expresses the conditional dependence structure between random variables

A **Bayesian network** is a probabilistic graphical model for representing knowledge about an uncertain domain where each node corresponds to a random variable and each edge represents the conditional probability for the corresponding random variables.

$$P(X) = \sum_{c \in C} P(c) P(X \mid c)$$

Probability of data

Probability of class

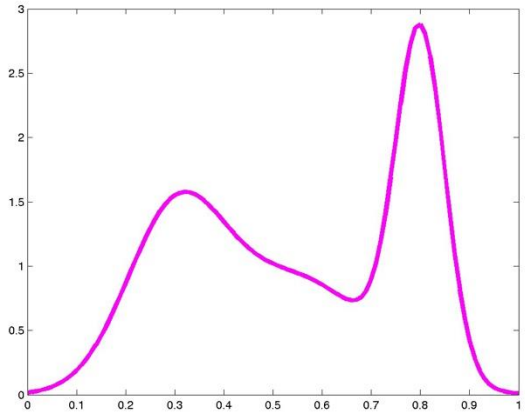


$P(c_1)=a,\ P(c_2)=b, \ldots$

C	P(X|c)
c_1	$N(\mu_1, \sigma_1)$
c_2	$N(\mu_2, \sigma_2)$
…	…

Learning MOGs

- We only have data like this:



- How do we learn which Gaussian each data point belongs to?
  - **Expectation-maximization algorithm**

Expectation-maximization (EM) algorithm

- Two parts, iterated over and over again until convergence
- Part 1: Expectation
  - What's our best guess for every data point as to which Gaussian cluster(mixture) it comes from?
  - In general, compute the probability of hidden variables
- Part 2: Maximization:
  - Given our expectations, figure out the parameters for the Gaussian distributions
  - In general, compute new parameters based on the probability of the hidden variables

Expectation-maximization (EM) algorithm

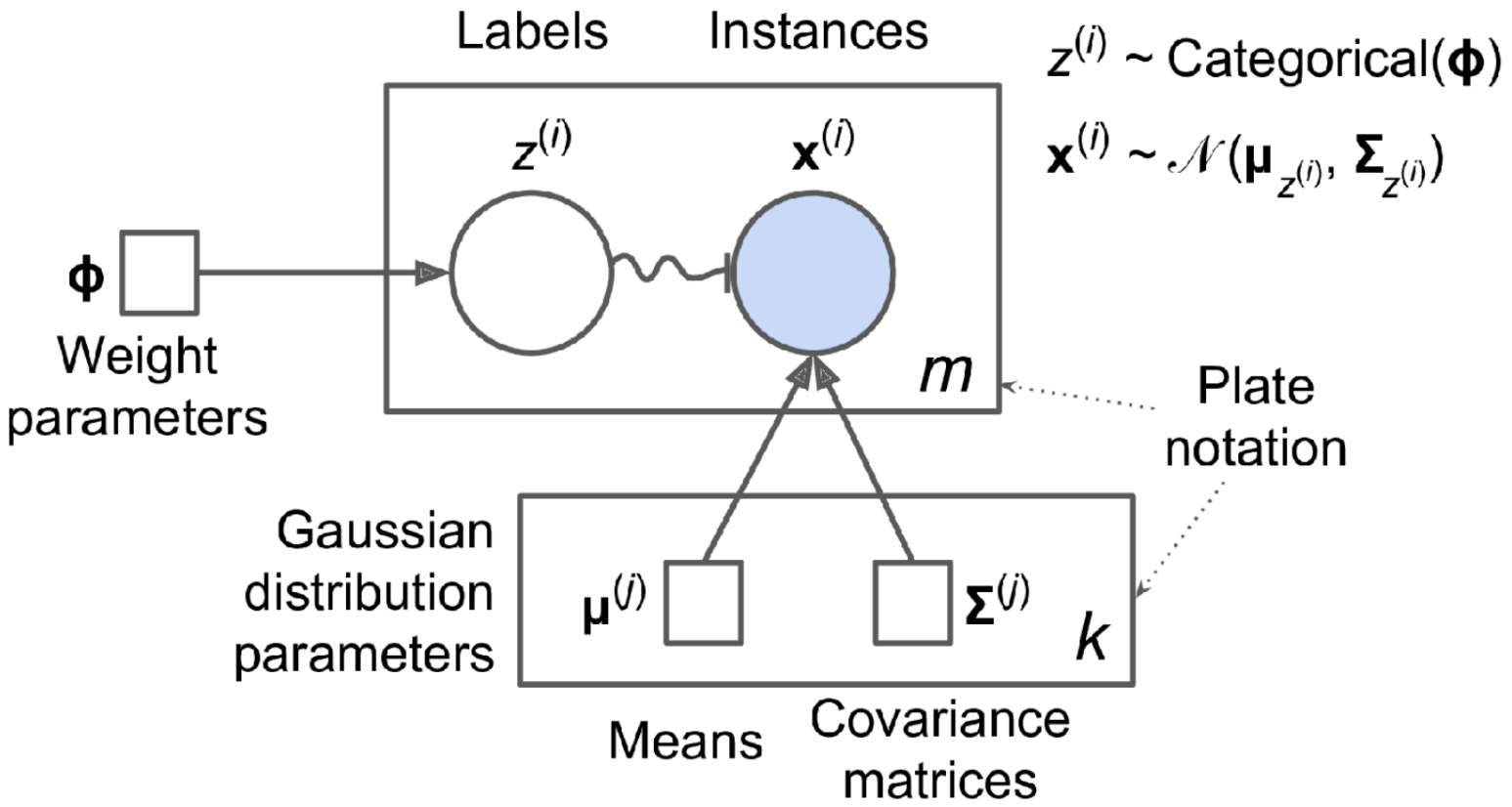


Figure 9-16. A graphical representation of a Gaussian mixture model, including its parameters (squares), random variables (circles), and their conditional dependencies (solid arrows)

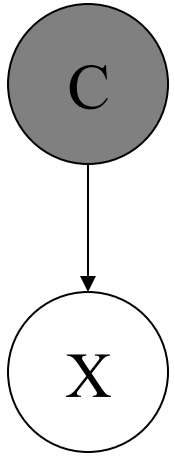
MOG: Initialization

- Set $P(c_i)$ to random values
  - Make sure the sum of $P(c_i)$'s over all i's, sums to 1 (e.g. $\sum_i P(c_i) = 1$)
- Set means and standard deviations for each Gaussian to random values
  - Helps to get them started in the right area

$$P(X) = \sum_{c \in C} P(c) P(X \mid c)$$

Probability of data

Probability of class



$P(c_1)=a,\ P(c_2)=b, \ldots$

C	P(X|c)
c_1	$N(\mu_1, \sigma_1)$
c_2	$N(\mu_2, \sigma_2)$
…	…

MOG: Expectation

- **Question**: for every point X_j, what is the probability that class i generated that point?

$$P_{ij}(c_i|X_j)$$

Reminder: Bayes theorem

- Joint, conditional and marginal probabilities:

$$P(A,B) = P(B|A)P(A) = P(A|B)P(B)$$

Joint probability A and B

Conditional probability B given A

Marginal probability

```
A = role an even die
B = rolling 6
```

$$P(B|A) = \frac{P(\{6\})}{P(\{2,4,6\})} = \frac{1/6}{3/6} = \frac{1}{3}$$

- Bayes theorem:

Likelihood

Prior distribution

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

Posterior distribution

MOG: Expectation

- **Question**: for every point X_j, what is the probability that class i generated that point?

$$P_{ij}(c_i|X_j) = \frac{P(X_j|c_i)P(c_i)}{P(X_j)}$$
$$= \frac{P(X_j|c_i)P(c_i)}{\sum_{c \in C} P(c_i)P(X_j|c_i)}$$

Effective number of points assigned to cluster *i*

$$N_i = \sum_j P_{ij}(c_i|X_j)$$

MOG: Maximization

- For every class, compute a new class mean, standard deviation, and prior

$$\hat{\mu}_i = \frac{\sum_j P_{ij} x_j}{N_i}$$

new mean: weighted average of points assigned to class *i*

$$\hat{\sigma}_i = \sqrt{\frac{\sum_j P_{ij} x_j^2}{N_i} - \left(\frac{\sum_j P_{ij} x_j}{N_i}\right)^2}$$

new standard deviation: calculated in same weighted manner

$$\hat{P}(c_i) = \frac{N_i}{\sum_j N_j}$$

new class prior: proportion of weighted samples attributed to class

MOG: Iterations of EM algorithm

- Repeat this process until convergence (more about this later)

Expectation

$$P_{ij}(c_i|X_j) = \frac{P(X_j|c_i)P(c_i)}{\sum_{c \in C} P(c_i)P(X_j|c_i)}$$

$$N_i = \sum_j P_{ij}(c_i|X_j)$$

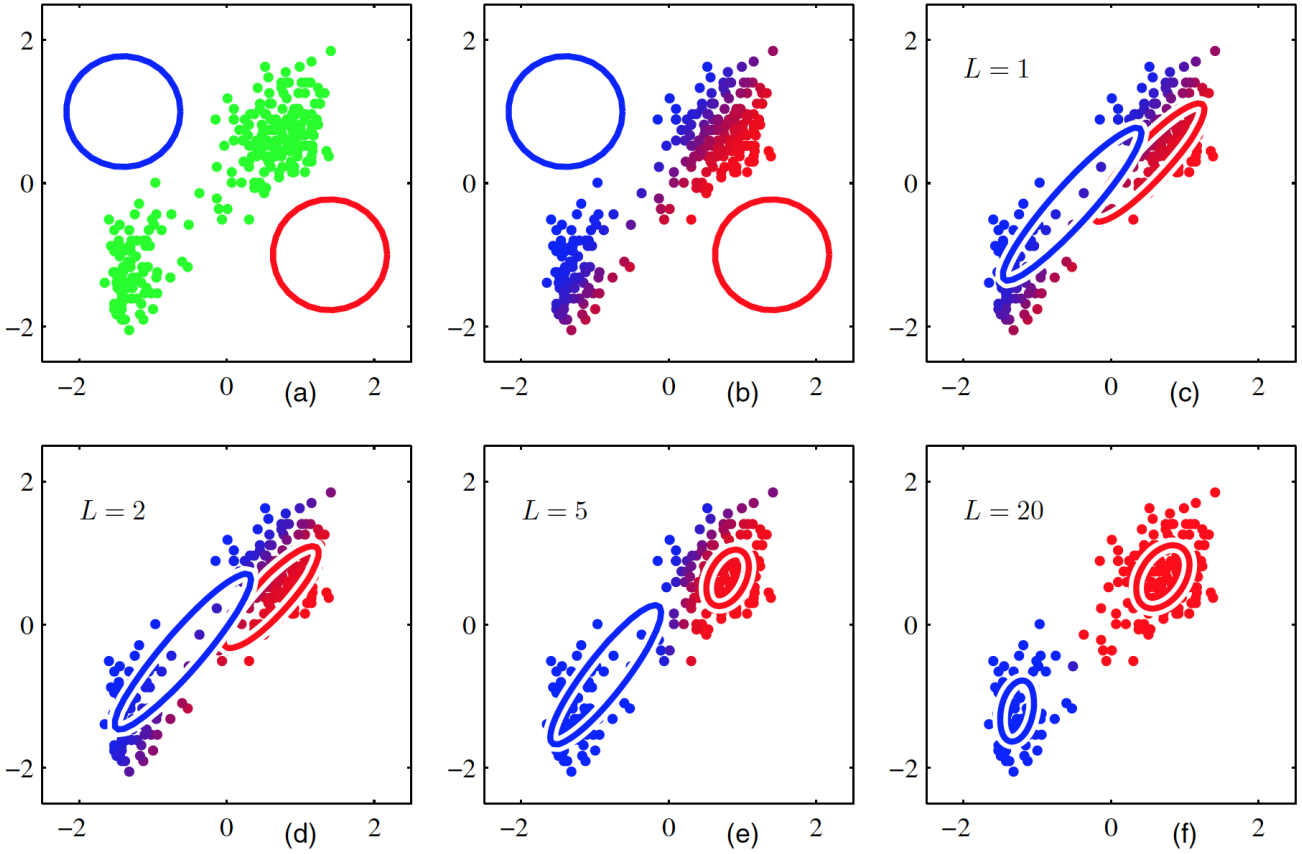
Maximization

$$\hat{\mu}_i = \frac{\sum_j P_{ij} x_j}{N_i}$$

$$\hat{\sigma}_i = \sqrt{\frac{\sum_j P_{ij} x_j^2}{N_i} - \left(\frac{\sum_j P_{ij} x_j}{N_i}\right)^2}$$

$$\hat{P}(c_i) = \frac{N_i}{\sum_j N_j}$$

MOG: Pictorial Training Example



Bishop, chapter 9

https://www.microsoft.com/en-us/research/people/cmbishop/prml-book/

EM in scikit learn

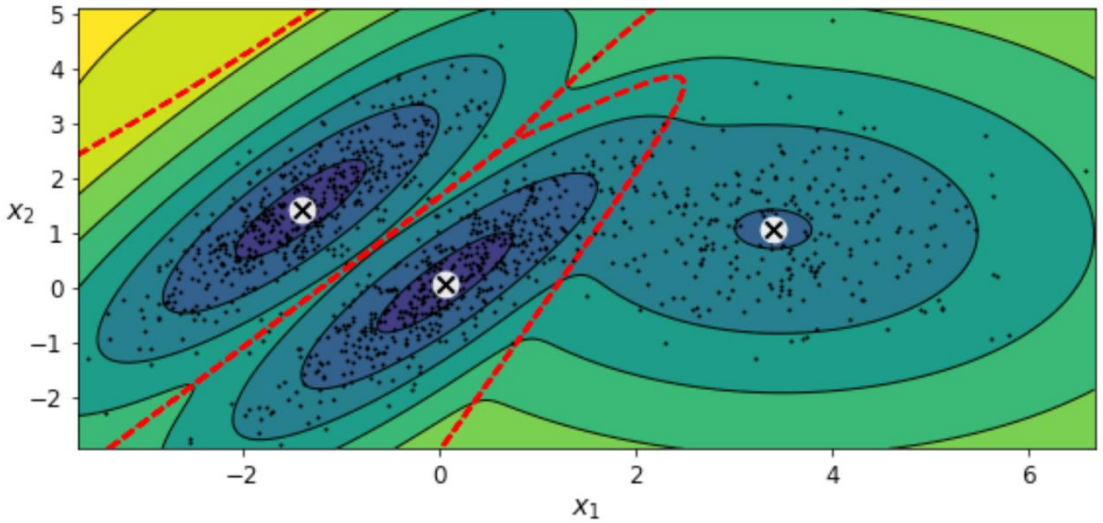
```
X1, y1 = make_blobs(n_samples=1000, centers=((4, -4), (0, 0)), random_state=42)
X1 = X1.dot(np.array([[0.374, 0.95], [0.732, 0.598]]))
X2, y2 = make_blobs(n_samples=250, centers=1, random_state=42)
X2 = X2 + [6, -8]
X = np.r_[X1, X2]
y = np.r_[y1, y2]
```

```
from sklearn.mixture import GaussianMixture
```

```
gm = GaussianMixture(n_components=3, n_init=10, random_state=42)
gm.fit(X)
```

```
gm.weights_
```

```
array([0.39054348, 0.2093669 , 0.40008962])
```




```
gm.means_
```

```
array([[ 0.05224874,  0.07631976],
       [ 3.40196611,  1.05838748],
       [-1.40754214,  1.42716873]])
```

```
gm.covariances_
```

```
array([[[ 0.6890309 ,  0.79717058],
        [ 0.79717058,  1.21367348]],

       [[ 1.14296668, -0.03114176],
        [-0.03114176,  0.9545003 ]],

       [[ 0.63496849,  0.7298512 ],
        [ 0.7298512 ,  1.16112807]]])
```

EM in scikit learn

- We can now use the model to predict which cluster each instance belongs to (hard clustering) or the probabilities that it came from each cluster:

```
gm.predict(X)
```

```
array([0, 0, 2, ..., 1, 1, 1])
```

```
gm.predict_proba(X)
```

```
array([[9.77227791e-01, 2.27715290e-02, 6.79898914e-07],
       [9.83288385e-01, 1.60345103e-02, 6.77104389e-04],
       [7.51824662e-05, 1.90251273e-06, 9.99922915e-01],
       ...,
       [4.35053542e-07, 9.99999565e-01, 2.17938894e-26],
       [5.27837047e-16, 1.00000000e+00, 1.50679490e-41],
       [2.32355608e-15, 1.00000000e+00, 8.21915701e-41]])
```

EM and convergence

- Unfortunately, just like K-Means, EM can end up converging to poor solutions, so it needs to be run several times, keeping only the best solution.
- When there are many dimensions, or many clusters, or few instances, EM can struggle to converge to the optimal solution. You might need to reduce the difficulty of the task by limiting the number of parameters that the algorithm has to learn.
  - One way to do this is to limit the range of shapes and orientations that the clusters can have.

`"spherical"`

All clusters must be spherical, but they can have different diameters (i.e., different variances).

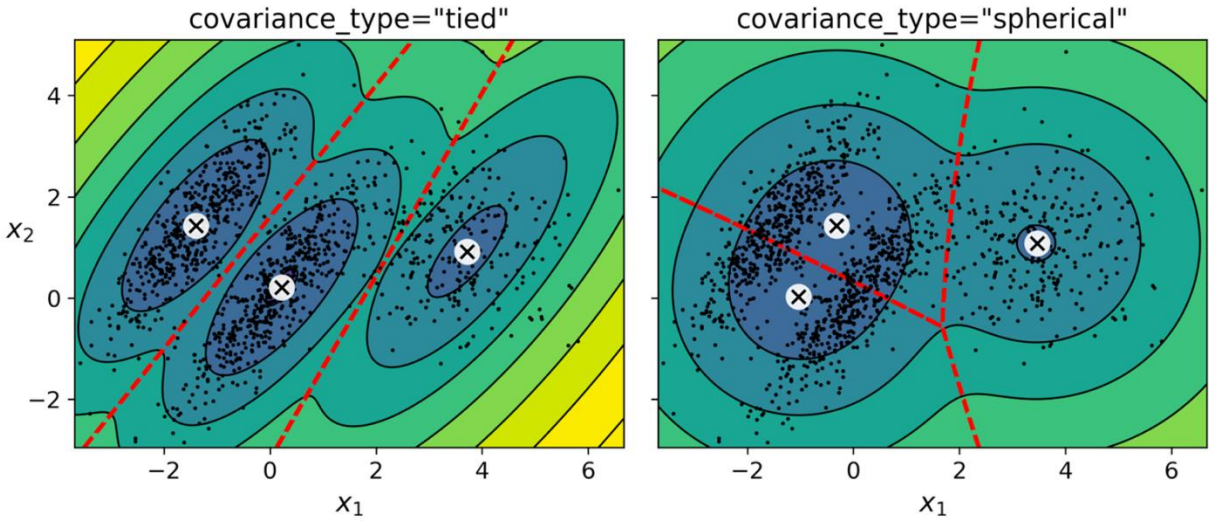
`"diag"`

Clusters can take on any ellipsoidal shape of any size, but the ellipsoid's axes must be parallel to the coordinate axes (i.e., the covariance matrices must be diagonal).

`"tied"`

All clusters must have the same ellipsoidal shape, size, and orientation (i.e., all clusters share the same covariance matrix).

EM and convergence



`"spherical"`

All clusters must be spherical, but they can have different diameters (i.e., different variances).

`"diag"`

Clusters can take on any ellipsoidal shape of any size, but the ellipsoid's axes must be parallel to the coordinate axes (i.e., the covariance matrices must be diagonal).

`"tied"`

All clusters must have the same ellipsoidal shape, size, and orientation (i.e., all clusters share the same covariance matrix).

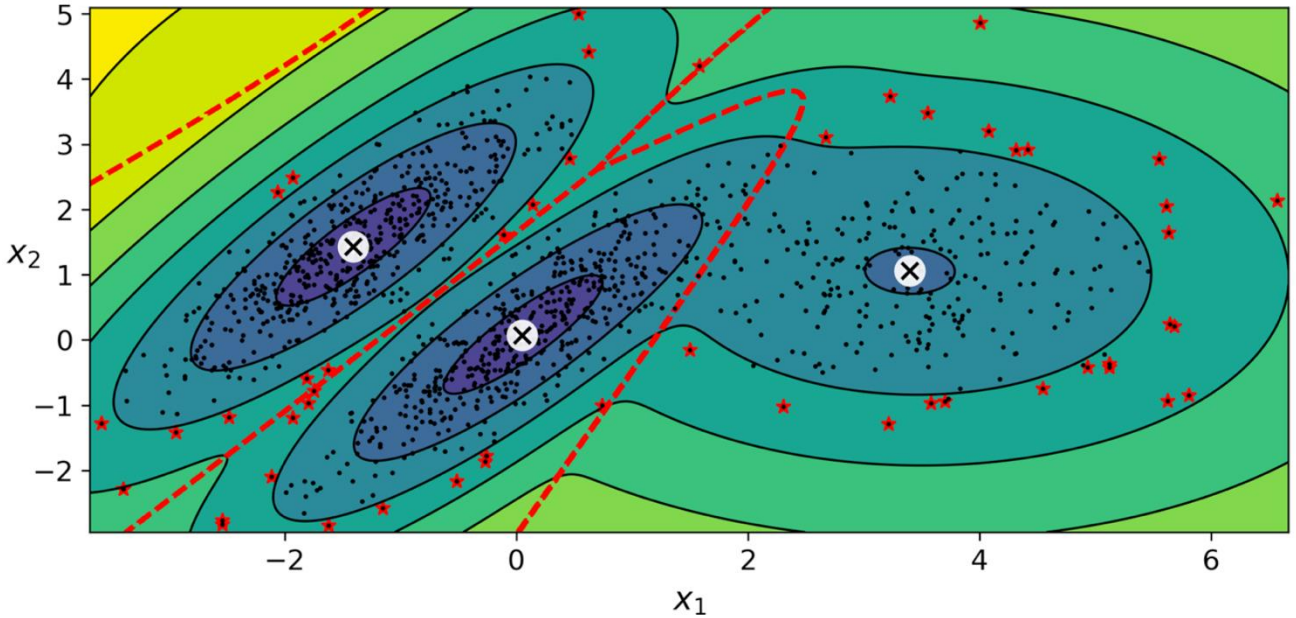
Anomaly Detection Using Gaussian Mixtures

- Anomaly detection (also called outlier detection) is the task of detecting instances that deviate strongly from the norm.
- These instances are called anomalies, or outliers, while the normal instances are called inliers.
- Anomaly detection is useful in a wide variety of applications, such as
  - fraud detection,
  - detecting defective products in manufacturing,
  - removing outliers from a dataset before training another model.

Anomaly Detection Using Gaussian Mixtures

- Using a Gaussian mixture model for anomaly detection is quite simple: any instance located in a low-density region can be considered an anomaly.
- For example, in a manufacturing company that tries to detect defective products, the ratio of defective products is usually well know n. Say it is equal to 4%.
- You then set the density threshold to be the value that results in having 4% of the instances located in areas below that threshold density.

```
densities = gm.score_samples(X)
density_threshold = np.percentile(densities, 4)
anomalies = X[densities < density_threshold]
```



Selecting the Number of Clusters

- With K-Means, we could use the inertia or the silhouette score to select the appropriate number of clusters.
- But with Gaussian mixtures, it is not possible to use these metrics because they are not reliable when the clusters are not spherical or have different sizes.
- Instead, we can try to find the model that minimizes a theoretical information criterion, such as the Bayesian information criterion (BIC) or the Akaike information criterion (AIC),

$$BIC = \log(m)\,p - 2\log\left(\widehat{L}\right)$$

$$AIC = 2p - 2\log\left(\widehat{L}\right)$$

- m is the number of instances, as always.
- p is the number of parameters learned by the model.
- $\widehat{L}$ is the maximized value of the *likelihood function* of the model.

Selecting the Number of Clusters

- Both the BIC and the AIC penalize models that have more parameters to learn (e.g., more clusters) and reward models that fit the data well.
- They often end up selecting the same model. When they differ, the model selected by the BIC tends to be simpler (fewer parameters) than the one selected by the AIC, but tends to not fit the data quite as well (this is especially true for larger datasets).

```
>>> gm.bic(X)
8189.74345832983
>>> gm.aic(X)
8102.518178214792
```

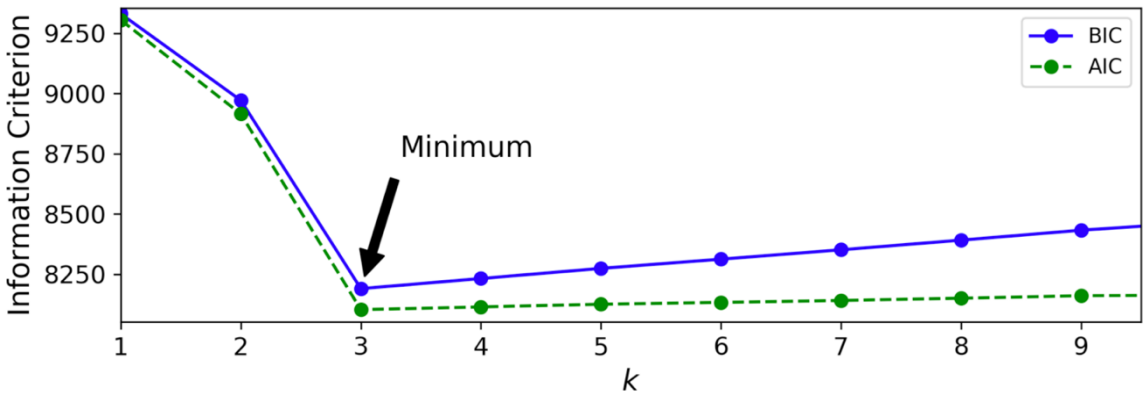


Figure 9-21. AIC and BIC for different numbers of clusters k

A note about Likelihood function

- The terms "probability" and "likelihood" are often used interchangeably in the English language, but they have very different meanings in statistics.
- Given a statistical model with some parameters θ:
  - the word "probability" is used to describe how plausible a future outcome x is (knowing the parameter values θ),
  - while the word "likelihood" is used to describe how plausible a particular set of parameter values θ are, after the outcome x is known.

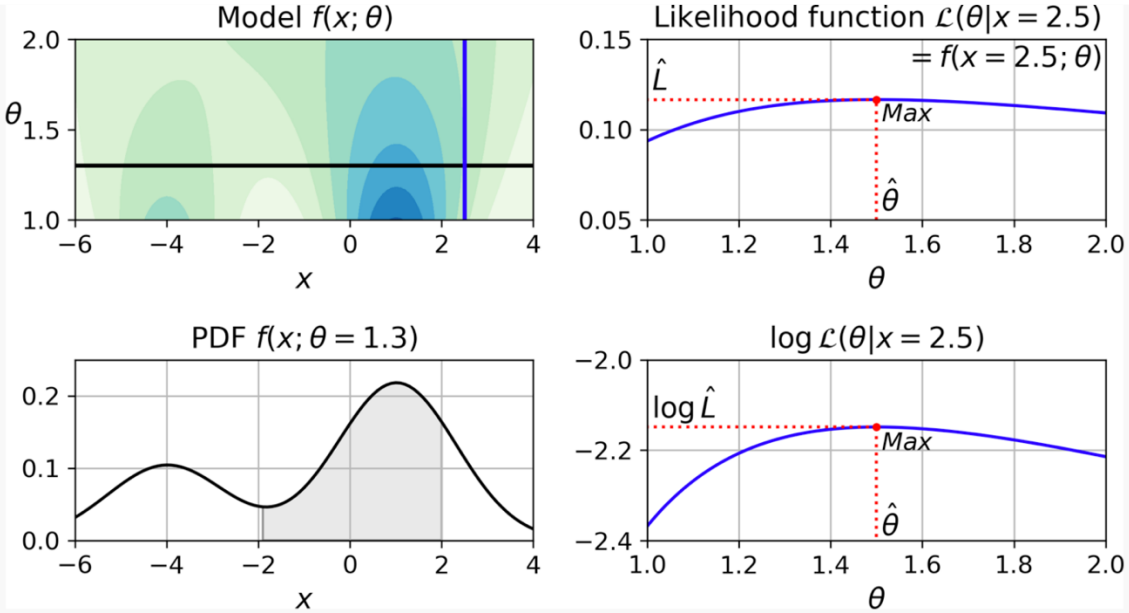


Figure 9-20. A model's parametric function (top left), and some derived functions: a PDF (lower left), a likelihood function (top right), and a log likelihood function (lower right)

Maximum Likelihood Estimate (MLE)

- Given a dataset **X**, a common task is to try to estimate the most likely values for the model parameters. To do this, we must find the values that maximize the likelihood function, given **X**.
- In this example, if we have observed a single instance x=2.5, the maximum likelihood estimate (MLE) of θ is 1.5.
- Notice that maximizing the likelihood function is equivalent to maximizing its logarithm (the logarithm is a strictly increasing function, so if θ maximizes the log likelihood, it also maximizes the likelihood).

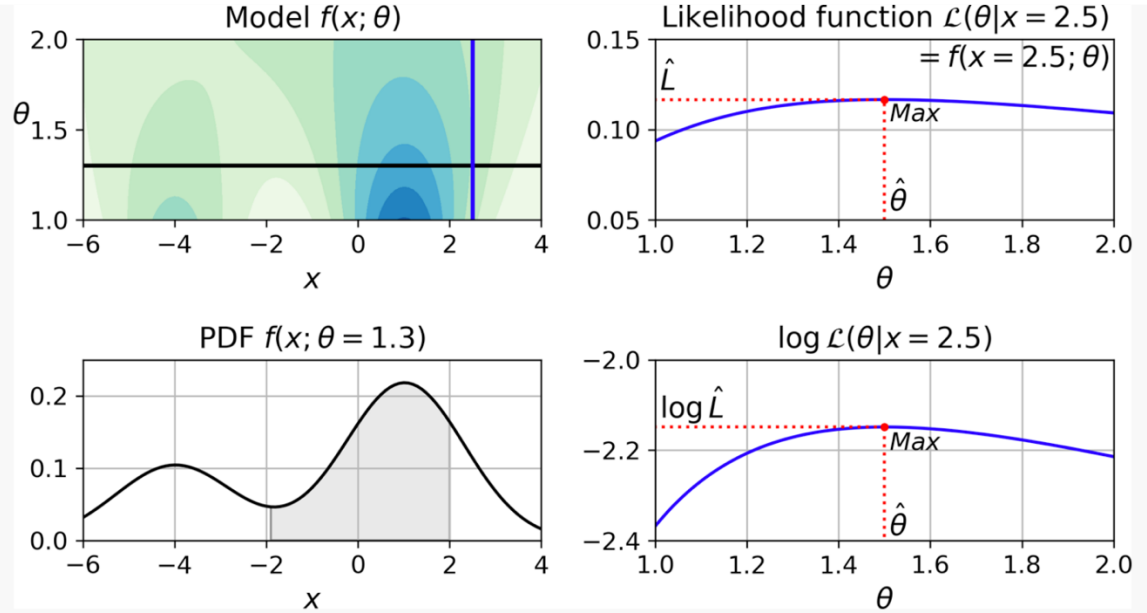


Figure 9-20. A model's parametric function (top left), and some derived functions: a PDF (lower left), a likelihood function (top right), and a log likelihood function (lower right)

Maximum Likelihood Estimate (MLE)

- It turns out that it is generally easier to maximize the log likelihood.
- For example, if you observed several independent instances $x^{(1)}$ to $x^{(m)}$, you would need to find the value of θ that maximizes the product of the individual likelihood functions.
- But it is equivalent, and much simpler, to maximize the sum (not the product) of the log likelihood functions, thanks to the magic of the logarithm which converts products into sums: $\log(ab)=\log(a)+\log(b)$.

Log-likelihood for MOG

- Therefore, the log likelihood is used to evaluate performance in EM
- For a set of data **X**, we define the log likelihood given *N(μ, σ)* as (assuming independence):

$$L = \sum_{j=1}^{N} \log\left(P(x_j)\right) = \sum_{j=1}^{N} \log\left(\sum_{i=1}^{I} P(c_i)\mathcal{N}(x_j|\mu_i, \sigma_i)\right)$$

where *N* is the number of data points, *I* is the number of Gaussian clusters, and x_j represents the jth data point. Note that $\mathcal{N}(x_j|\mu_i, \sigma_i)$ can be replaced with the equation for a Normal/Gaussian distribution with mean μ_i and standard deviation σ_i.

Maximum a-posteriori (MAP)

- If a prior probability distribution g over θ exists, it is possible to take it into account by maximizing (θ|x)g(θ) rather than just maximizing (θ|x).
- This is called maximum a-posteriori (MAP) estimation. Since MAP constrains the parameter values, we can think of it as a regularized version of MLE.

Bayes theorem

Likelihood

Prior distribution

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

Posterior distribution

Next time

- Introduction to Artificial Neural Networks with Keras, from Chapter 10 *Hands on machine learning textbook*.